

CS102L Data Structures & Algorithms

Lab 7



DEPARTMENT OF COMPUTER SCIENCE
DHANANI SCHOOL OF SCIENCE AND ENGINEERING
HABIB UNIVERSITY
SPRING 2026

Contents

1	Introduction	2
1.1	Instructions	2
1.2	Marking scheme	2
1.3	Lab Objectives	2
1.4	Late submission policy	2
1.5	Use of AI	3
1.6	Viva	3
2	Prerequisites	4
2.1	Pytest	4
3	Exercises	5
3.1	QuickSort Algorithm	5
3.1.1	Function Description	5
3.1.2	QuickSort Algorithm	5
3.1.3	Examples	6
3.1.4	Testing	6
3.2	Sort Matrix By Column Number	6
3.2.1	Function Description	6
3.2.2	Sample	7
3.2.3	Testing	7
3.3	Sorting Rectangle Records	7
3.3.1	Problem Description	7
3.3.2	Function Description	8
3.3.3	Sample	8
3.3.4	Testing	9

Introduction

1.1 Instructions

- This lab will contribute 1% towards your final grade.
- The deadline for submission of this lab is at the end of lab time.
- The lab must be submitted online via CANVAS. You are required to submit a zip file that contains all the *.py* files.
- The zip file should be named as *lab7-aa12345.zip* where *aa12345* will be replaced with your student id.
- **Files that don't follow the appropriate naming convention will not be graded.**
- **Your final grade will comprise of both your submission and your lab performance.**

1.2 Marking scheme

This lab will be marked out of 100.

- 50 Marks are for the completion of the lab.
- 50 Marks are for progress and attendance during the lab.

1.3 Lab Objectives

- To understand and implement quick sort, using different pivot selection schemes, in Python.
- To use quick sort in different exercises.

1.4 Late submission policy

There is no late submission policy for the lab.

1.5 Use of AI

Taking help from any AI-based tools such as ChatGPT is strictly prohibited and will be considered plagiarism.

1.6 Viva

Course staff may call any student for Viva to provide an explanation for their submission.

Prerequisites

2.1 Pytest

Pytest is a popular testing framework in Python known for its simplicity and power in facilitating efficient and scalable testing for applications and software. In order to use Pytest, one first needs to install it and create test functions prefixed with *test_* in a Python file. To install Pytest, you can write the following command on the terminal:

```
pip install pytest
```

After installing Pytest, one can execute all the tests by typing the following command on the terminal:

```
pytest
```

If you want to test any particular file, then you will have to specify the filename as follows:

```
pytest <filename>.py
```

In case the above mentioned pytest commands do not work on your system, even though pytest is installed, try using the following commands:

```
py -m pytest
```

```
py -m pytest <filename>.py
```

If all the tests pass successfully, then you should receive a similar output.

```
PS C:\Work\Spring2024\DSA\Lab1> pytest test_Question1.py
===== test session starts =====
platform win32 -- Python 3.11.2, pytest-7.4.4, pluggy-1.3.0
rootdir: C:\Work\Spring2024\DSA\Lab1
collected 10 items

test_Question1.py .....

===== 10 passed in 0.06s =====
```

However, if all the tests fail, then you should receive a similar output.

```
PS C:\Work\Spring2024\DSA\Lab1> pytest test_Question1.py
===== test session starts =====
platform win32 -- Python 3.11.2, pytest-7.4.4, pluggy-1.3.0
rootdir: C:\Work\Spring2024\DSA\Lab1
collected 10 items

test_Question1.py FFFFFFFF

===== FAILURES =====
test_question1[1st10-1st20-e54dc15ccafa608142ffa6340b035c327f972e24882052cfc3345f240f4ee237] ____
```

Exercises

3.1 QuickSort Algorithm

This function should be implemented in the file `q1.py`.

3.1.1 Function Description

Write a function named `quick_sort`, which takes the following parameters:

- `lst` – An unsorted list of elements.
- `low` – The index of the first element of the current sort range.
- `high` – The index of the last element of the current sort range.

The function should sort `lst` in-place using the quicksort algorithm, using the **middle element** as the pivot. The function should not return anything. After partitioning and before making any recursive calls, the current state of `lst` should be printed.

3.1.2 QuickSort Algorithm

You can implement the Quick Sort Algorithm by following the given pseudocode:

Pseudocode 1 QuickSort Algorithm

Require: A list *lst* and indices *low* and *high* indicating the portion to sort

- 1: **if** `low < high` **then**
 - 2: Get partitioning point of the array by calling partition function
 - 3: Call QuickSort for first partition.
 - 4: Call QuickSort for second partion.
 - 5: **end if**
-

Pseudocode 2 Pivot Selection Algorithm

Require: *lst*: list to partition, *low*: starting index, *high*: ending index

Ensure: index of the *pivot* element after moving it to the start

- 1: Set *pivot* according to pivot selection scheme
 - 2: Swap values at index *pivot* and *low* so that pivot becomes the low index.
 - 3: **return** *low*
-

Pseudocode 3 Partition Function

Require: *lst*: list to partition, *low*: starting index, *high*: ending index

Ensure: index of the *pivot* element

- 1: Call Pivot Selection function to get index of the *pivot* element.
 - 2: Set *low* to *low*+1.
 - 3: Increment *low* until you find a *value*(at *low* index) > *pivot* element and *low* <= *high*
 - 4: Decrement *high* until you find a *value*(at *high* index) <= *pivot* element and *low* <= *high*
 - 5: if *low* <= *high*: Swap values at index *low* and *high*
 - 6: Continue while *low* <= *high*
 - 7: Swap values at index *pivot* and *high*
 - 8: **return** *high*
-

3.1.3 Examples

```
>>> lst = [5, 9, 2, 1, 6, 3, 0]
>>> quick_sort(lst, 0, len(lst) - 1)
[0, 1, 2, 5, 6, 3, 9]
[0, 1, 3, 5, 2, 6, 9]
[0, 1, 2, 3, 5, 6, 9]
[0, 1, 2, 3, 5, 6, 9]

>>> lst = [21, 36, 11, 9, 6, 42, 39]
>>> quick_sort(lst, 0, len(lst) - 1)
[6, 9, 11, 21, 36, 42, 39]
[6, 9, 11, 21, 36, 42, 39]
[6, 9, 11, 21, 36, 42, 39]
[6, 9, 11, 21, 36, 39, 42]
```

3.1.4 Testing

To test your function, type one of the following commands in the terminal:

```
pytest tests/test_q1.py
```

3.2 Sort Matrix By Column Number

This function should be implemented in the file **q2.py**.

3.2.1 Function Description

Write a function named `quick_sort_by_column_number`, which takes the following parameters:

- `matrix` – A 2D list (list of lists) where each inner list represents a row.

- `low` – The index of the first row of the current sort range.
- `high` – The index of the last row of the current sort range.
- `column_number` – The index of the column by which the rows should be sorted.

The function should sort `matrix` in-place by the values in the specified `column_number`, using a modified QuickSort algorithm that uses the **last element** (highest index) as the pivot. The function should not return anything. After partitioning and before making any recursive calls, the current state of `matrix` should be printed.

3.2.2 Sample

```
>>> matrix = [[7, 8, 9], [1, 2, 3], [4, 5, 6]]
>>> quick_sort_by_column_number(matrix, 0, 2, 2)
[[1, 2, 3], [4, 5, 6], [7, 8, 9]]

>>> matrix = [['square', 'rectangle', 'triangle'], ['chair',
    'table', 'house'], ['motor cycle', 'car', 'truck']]
>>> quick_sort_by_column_number(matrix, 0, 2, 2)
[['square', 'rectangle', 'triangle'], ['chair', 'table', '
    house'], ['motor cycle', 'car', 'truck']]
[['chair', 'table', 'house'], ['square', 'rectangle', '
    triangle'], ['motor cycle', 'car', 'truck']]
```

3.2.3 Testing

In order to test your function, type the following command on the terminal:

```
pytest tests/test_q2.py
```

3.3 Sorting Rectangle Records

This function should be implemented in the file `q3.py`.

3.3.1 Problem Description

Suppose that we have the following rectangle records stored as a list of dictionaries:

```
rectangle_records = [
    {"ID": "Rect1", "Length": 40, "Breadth": 25, "Color": "
    red"},
    {"ID": "Rect2", "Length": 30, "Breadth": 20, "Color": "
    blue"},
    {"ID": "Rect3", "Length": 70, "Breadth": 45, "Color": "
    green"}]
```

```

    {"ID": "Rect4", "Length": 20, "Breadth": 10, "Color": "
    purple"}
]

```

You are required to sort `rectangle_records` in *ascending order* by a specified `record_title`. The value of `record_title` will be one of *"ID"*, *"Length"*, *"Breadth"*, or *"Color"*.

3.3.2 Function Description

Write a function named `quick_sort_rectangles`, which takes the following parameters:

- `rectangle_records` – A list of dictionaries, where each dictionary represents a rectangle record with keys *"ID"*, *"Length"*, *"Breadth"*, and *"Color"*.
- `low` – The index of the first element of the current sort range.
- `high` – The index of the last element of the current sort range.
- `record_title` – The dictionary key by which the records should be sorted.

The function should sort `rectangle_records` in-place in *ascending order* by `record_title`, using a modified QuickSort algorithm that uses the **first element** (lowest index) as the pivot. The function should not return anything. After partitioning and before making any recursive calls, the current state of `rectangle_records` should be printed.

3.3.3 Sample

```

>>> rectangle_records = [{"ID": "Rect1", "Length": 40, "
    Breadth": 25, "Color": "red"}, {"ID": "Rect2", "Length":
    30, "Breadth": 20, "Color": "blue"}, {"ID": "Rect3", "
    Length": 70, "Breadth": 45, "Color": "green"}, {"ID": "
    Rect4", "Length": 20, "Breadth": 10, "Color": "purple"}]

>>> quick_sort_rectangles(rectangle_records, 0, len(
    rectangle_records)-1, "Length")
[{'ID': 'Rect4', 'Length': 20, 'Breadth': 10, 'Color': '
    purple'}, {'ID': 'Rect2', 'Length': 30, 'Breadth': 20, '
    Color': 'blue'}, {'ID': 'Rect1', 'Length': 40, 'Breadth':
    25, 'Color': 'red'}, {'ID': 'Rect3', 'Length': 70, '
    Breadth': 45, 'Color': 'green'}]
[{'ID': 'Rect4', 'Length': 20, 'Breadth': 10, 'Color': '
    purple'}, {'ID': 'Rect2', 'Length': 30, 'Breadth': 20, '
    Color': 'blue'}, {'ID': 'Rect1', 'Length': 40, 'Breadth':
    25, 'Color': 'red'}, {'ID': 'Rect3', 'Length': 70, '
    Breadth': 45, 'Color': 'green'}]

```

3.3.4 Testing

In order to test your function, type the following command on the terminal:

```
pytest tests/test_q3.py
```